Lab 1

Python and the Census

Introduction to Pandas, Matplotlib, and the Census API

DATA 20195 Spring 2026

Emma Ross and Michael Xiao

Announcements

- Welcome to lab!
- **Homework 1 due Wednesday, April 1st, 11:59 pm**, with automatic extension to Friday, April 3rd, 1:30pm *(read syllabus for details)*
- Readings
  - **Week 1:** James Scott, Seeing Like a State, Introduction; Melissa Nobles, Shades of Citizenship, Chapter 1: Race, Censuses, and Citizenship
  - **Week 2:** Ian Hacking, Kinds of People: Moving Targets
  - Printed copies available up front
  - Further optional readings available on Github
- Office Hours start this coming week, March 30th – April 3rd
  - Emma: Tuesdays, 2–3pm
  - Michael: Thursdays, 2–3pm

Locations announced Monday in class and by email!

Logistics

- Lab attendance is **required**. If you miss a lab, you must submit the lab absence form.
- Please attend the lab you signed up for.
- Assignment submission: Each week, you will submit…
  - Lab turn-in: via **Google Form** by end of lab
  - Homework: submitted via **Gradescope** by following Wednesday.
    - Code: **ZJ5X4E**

Collab Basics Demo

Intro to Python and Packages

Slides adapted from UC Berkeley's Data 100 (Principles and Techniques of Data Science) Summer 2025

“Tabular data” = data in a table

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

A row represents one observation (here, a single course offered this quarter).

A column represents some characteristic, or feature, of that observation (here, the instructor for the course).

The Standard Python Data Science Tool: `pandas`

`pandas` (stands for "panel data") is the standard tool across research and industry for working with tabular data.

Using `pandas`, we can:

- **Arrange** data in a tabular format.
- **Extract** useful information filtered by specific conditions.
- **Operate** on data to gain new insights.

Image credit: Wikipedia

DataFrame

Tabular data in `pandas` is referred to as `DataFrame`.

Many approaches exist for creating a `DataFrame`. Here, we will go over the most popular ones.

- From an **external** file (e.g. CSV, TSV, JSON).
- Using a **list** and column name(s).
- From a **dictionary**.

How we made our example `DataFrame`:

```
df = pd.DataFrame(
    {
        "id": [
            "CAPP 30235",
            "DATA 30195",
            "CAPP 30254",
            "ECON 56300",
        ],
        "name": [
            "Databases for Public Policy",
            "Data and the State",
            "Machine Learning for Public Policy",
            "Public Policy and Economics Workshop",
        ],
        "instructor": ["Raul Castro Fernandez", "Moon Duchin", "Hannah Morgan", "Dmitri  Koustas"],
        "credits": [3.0, 3.0, 3.0, 4.0],
        "enrollment_cap": [32, 50, 23, 56],
    }
)
```

Row and Column labels of a `DataFrame`

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

Columns of a DataFrame are labeled by **column names**.

Rows of a DataFrame are labeled by **row names** or **indices**.

Selecting Row(s) or Column(s)

Row and column labels allow us to access a “slice” of the `DataFrame`

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

```
df[[“name”,“credits”]]
```

	name	credits
0	Databases for Public Policy	3.0
1	Data and the State	3.0
2	Machine Learning for Public Policy	3.0
3	Public Policy and Economics Workshop	4.0

Selecting Row(s) or Column(s)

Row and column labels allow us to access a "slice" of the `DataFrame`

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

```
df.loc[[0,1,3]]
```

	id	name	professor	credits	enrollment_cap	workload
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32	10
1	DATA 30195	Data and the State	Moon Duchin	3.0	50	12
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56	12

Utility Functions

`pandas` also provides an enormous number of useful utility functions, for example:

```
df.head(2)
```

	id	name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50

```
df.sort_values(by="enrollment_cap", ascending=True)
```

	id	name	instructor	credits	enrollment_cap
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

Utility Functions: method chaining

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

```
df.sort_values(by="enrollment_cap", ascending=True).head(2)
```

	id	name	instructor	credits	enrollment_cap
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32

Utility Functions: ordering

	id	course_name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50
2	CAPP 30254	Machine Learning for Public Policy	Hannah Morgan	3.0	23
3	ECON 56300	Public Policy and Economics Workshop	Dmitri Koustas	4.0	56

What happens if we do…

```
df.head(2).sort_values(by="enrollment_cap", ascending=True)
```

	id	name	instructor	credits	enrollment_cap
0	CAPP 30235	Databases for Public Policy	Raul Castro Fernandez	3.0	32
1	DATA 30195	Data and the State	Moon Duchin	3.0	50

Python Plotting Tool: `matplotlib`

`matplotlib` (a combination of MATLAB, plot, and library) is a standard Python library for basic plotting.

Using `matplotlib`, we will create:

- **Histograms**
- **Scatter plots**
- **Line graphs**
- **Pie chart**

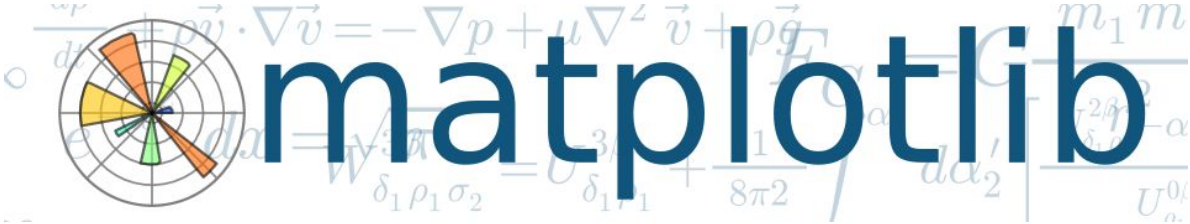
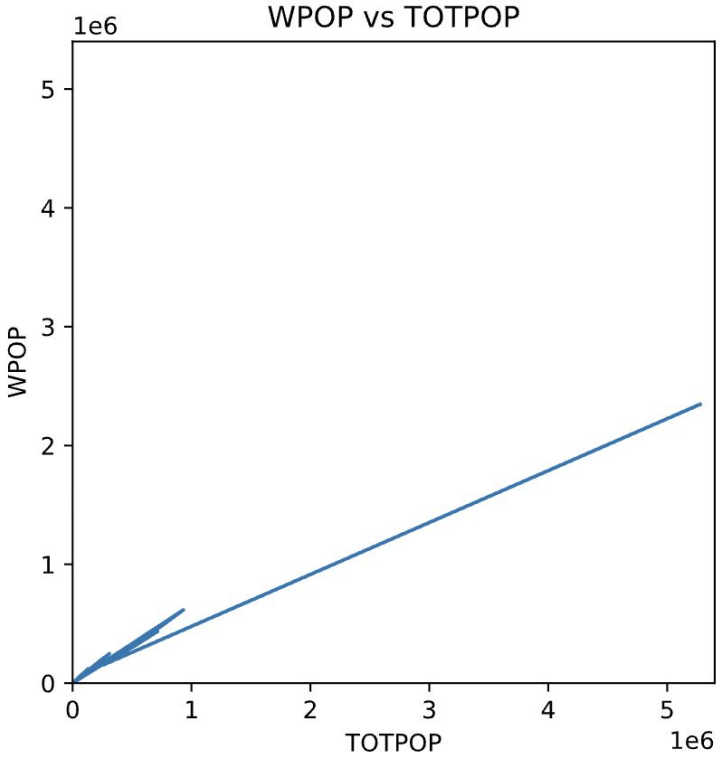
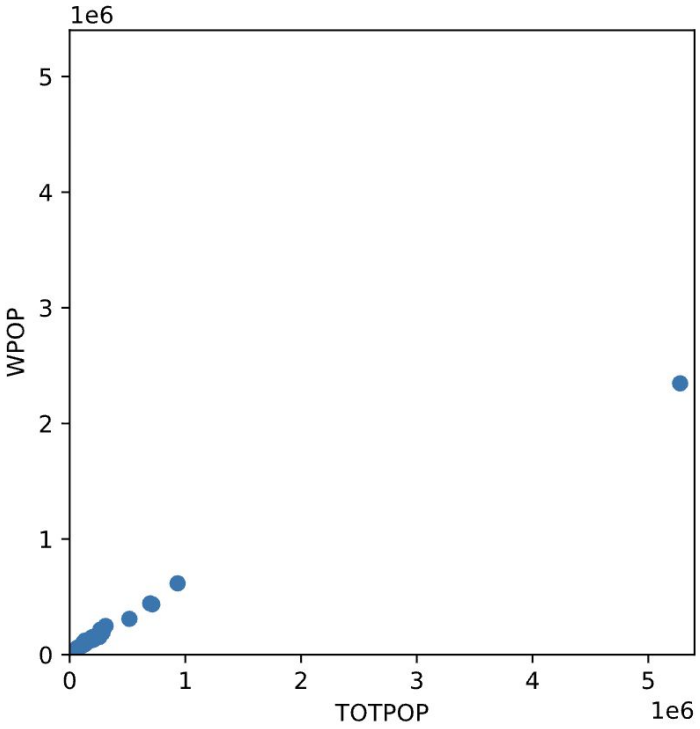


Image credit: Wikipedia

Plotting best practices

What type of plot best communicates this information?





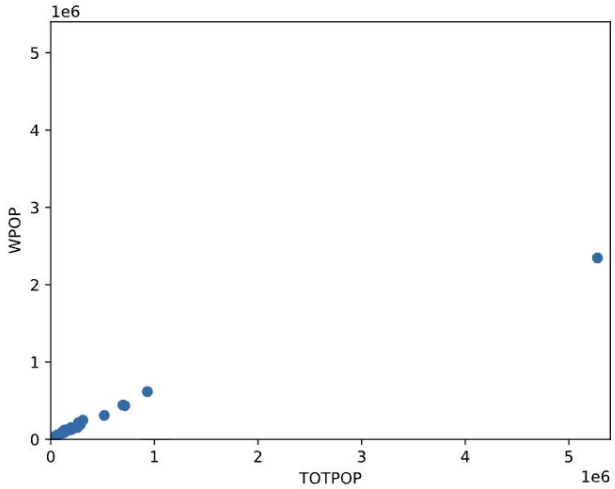
Plotting best practices

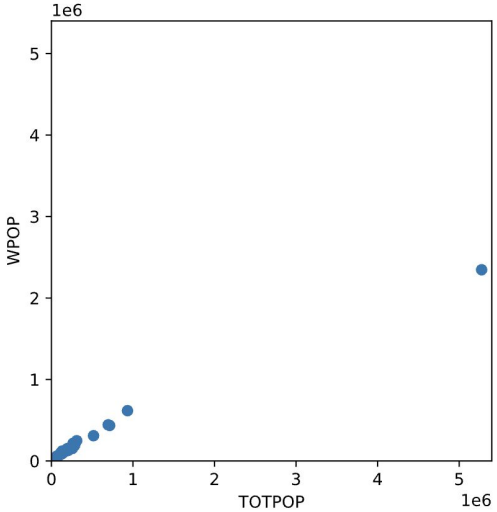
Compare common *types* of information:

- counts vs. counts
- percentages vs. percentages

Pay attention to your axis dimensions

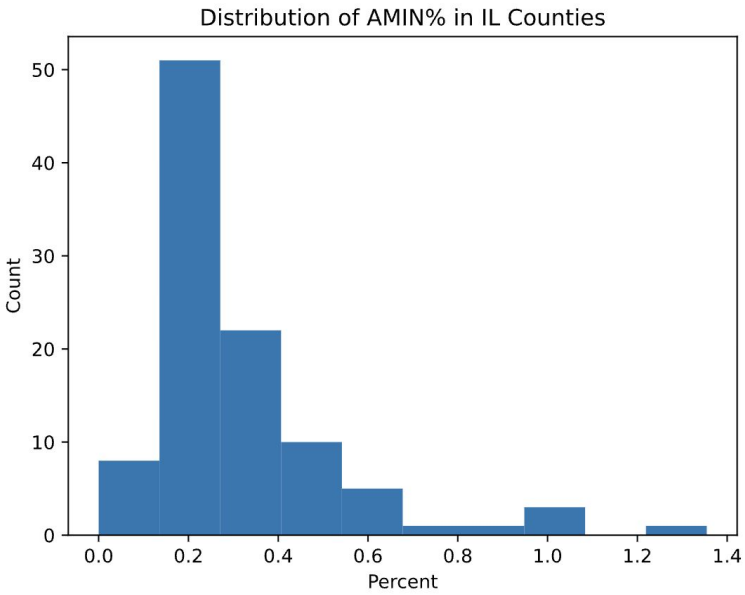
```
ax.set_aspect("equal")
```





Plotting best practices

Set informative/interpretable bins in your histograms when needed.



Accessing data through the Census API

What is an **API** (Application Programming Interface)?

- acts as a middleman between two programming systems to securely and consistently share data

What packages do we use to access data through the Census API?

```
!pip install -q census us
from census import Census
from us import states
```

`census` package is a wrapper around the US Census API

`us` package has state names and abbreviations

Census Data

- First U.S. census was in 1790, and have been taken decennially ever since.
- By using the Census API, there are **many datasets** you can access.
- For this lab, we focus on Table P1.
- Official column names used by the Census Bureau are frequently unintelligible — reference API documentation for help.

`P1_003N`: White alone

`P1_004N`: Black or African American alone

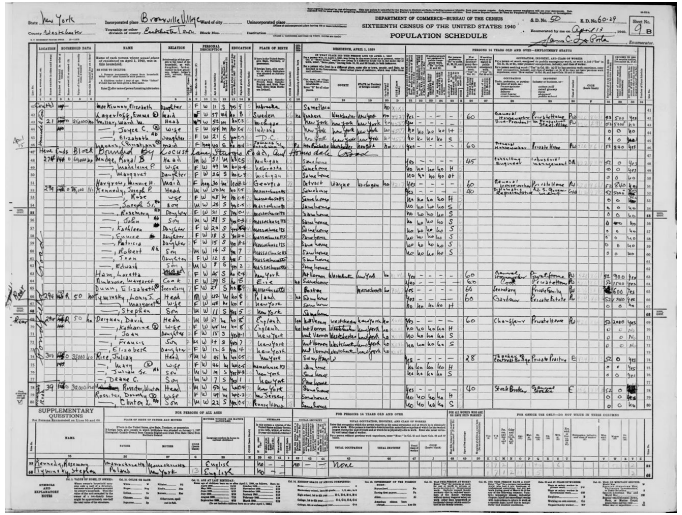
`P1_005N`: American Indian and Alaska Native alone

`P1_006N`: Asian alone

`P1_007N`: Native Hawaiian and Other Pacific Islander alone

`P1_008N`: Some Other Race alone

`P1_009N`: Two or more races



1940 Census Population table from Westchester, NY

https://tinyurl.com/20195-lab1